

# Global Guarded Vector Hue 位置指纹通信实验流程说明

## Position Fingerprint Experiment

2026 年 5 月 14 日

### 摘要

本文档根据 `vector_hue/global_guarded_core.py`、`virtual_stream_core.py` 和当前实验输出文件整理完整实验流程。实验目标不是给设备人工分配正交码，而是从不同空间位置的实测响应中提取位置指纹；发送端把多路 bit 组合映射为四通道 vector hue；合法设备使用自己的位置指纹正确恢复自己的信息；非法设备即使用自己的位置指纹尝试解码，也只能得到接近随机的结果。文档包含基于真实数据的完整例子：合法位置为 (2, 5, 26)，候选编号为 267，真实 Probe 集合为 [25, 75, 100, 115, 180, 215, 225, 250, 270, 295, 315, 325, 335, 340, 355]。

## 1 一句话概括

当前实验可以概括为：从每个位置的实测 Probe 响应中提取本地地址码，发送端用 **vector hue mapping** 同时承载多路信息，合法位置用自己的地址码做相关解码，非法位置由于指纹不同而无法稳定恢复任一路合法信息。

与 PDF 1 中的两位置最小 PoC 相比，当前代码已经扩展为更完整的安全实验：

- (1) 支持  $k$  个真实合法位置，本例  $k = 3$ ；
- (2) 通过虚拟流把有效维度扩展到 `target_effective_k=8`；
- (3) 每个符号组合不再映射到单个 hue，而是映射到四个 chip 各自的 hue，即 vector hue；
- (4) 不只追求合法端正确，还显式搜索并修复非法端最容易泄露的 weak routes；
- (5) 最后用线性规划混合多个候选 schedule，使最坏非法路由也尽量接近随机猜测。

## 2 实验对象和基本名词

### 2.1 位置、Probe 和 chip

实验数据存放在 `data/15pro/yellow_shuffled`。每个 CSV 文件对应一个空间位置，例如位置 2、位置 5、位置 26。每一行对应一个 Probe hue，例如 25、75、100；每一行有 4 列数值，表示四个显色片或四个 chip 的响应。因此，对位置  $p$  和  $m$  个 Probe，可以得到响应矩阵

$$Y_p \in \mathbb{R}^{m \times 4}.$$

本例中  $m = 15$ ，四个 chip 分别记为 chip1 到 chip4。

## 2.2 合法设备和非法设备

一次实验选出一组合法位置

$$\mathcal{L} = \{\ell_1, \ell_2, \dots, \ell_k\}.$$

这些位置是目标接收者。其他位置视为非法位置

$$\mathcal{I} = \mathcal{P} \setminus \mathcal{L}.$$

本例中合法位置为

$$\mathcal{L} = \{2, 5, 26\}.$$

位置 25 是非法位置之一，后文用它举例说明非法解码失败。

## 2.3 真实流和虚拟流

`real_k=3` 表示真实需要服务的合法位置有 3 个。代码还设置

$$\text{target\_effective\_k} = 8,$$

所以会补充

$$\text{virtual\_stream\_count} = 8 - 3 = 5$$

个虚拟流。虚拟流不对应真实用户，它们的作用是增加符号组合空间，使非法端更难把自己的输出稳定对应到某一路真实 bit。

# 3 位置指纹如何从真实数据中长出来

## 3.1 线性去趋势

对每个位置  $p$ , 输入 Probe 序列  $x = [q_1, \dots, q_m]$  和四通道响应矩阵  $Y_p$ 。代码在 `extract_fingerprint` 中对每个 chip 单独拟合线性趋势：

$$Y_p[:, d] \approx a_{p,d}x + b_{p,d}.$$

然后得到残差矩阵

$$R_p = Y_p - T_p.$$

直观理解：线性趋势表示 hue 变大时响应整体变大的公共部分；残差表示这个位置独有的非线性起伏，这部分更像位置指纹。

## 3.2 SVD 提取读出方向

对残差矩阵做奇异值分解：

$$R_p = U\Sigma V^\top.$$

取第一右奇异向量作为读出方向：

$$w_p = V_1.$$

然后把每个 Probe 的四通道残差压缩成一维值：

$$z_p = R_p w_p.$$

最后取符号得到本地地址码：

$$c_p[t] = \begin{cases} +1, & z_p[t] \geq 0, \\ -1, & z_p[t] < 0. \end{cases}$$

### 3.3 真实数据例子：三个合法位置的指纹

本例使用候选 267 的 15 个 Probe：

$$[25, 75, 100, 115, 180, 215, 225, 250, 270, 295, 315, 325, 335, 340, 355].$$

从真实 CSV 中读取响应并计算后，得到如下位置指纹。

为避免长向量在表格中挤压，下面按位置分别列出。每个位置的  $w_p$  是四个 chip 的读出方向， $c_p$  是 15 个 Probe 对应的地址码， $z_p$  是残差投影值。

#### 位置 2

$$w_2 = [0.1300, 0.5312, -0.7659, -0.3380]$$

$$c_2 = [+,-,+,-,+,-,-,+,-,+,+,+,+,-]$$

$$\begin{aligned} z_2 \approx [213.356, -163.890, 127.067, -106.743, 28.668, \\ -200.094, -240.826, 100.507, -51.133, 110.077, \\ 165.079, 67.521, 21.722, 75.131, -146.441]. \end{aligned}$$

#### 位置 5

$$w_5 = [-0.6661, 0.2772, -0.5661, 0.3987]$$

$$c_5 = [+,+,-,-,+,+,-,-,+,-,+,+,-,-,+]$$

$$\begin{aligned} z_5 \approx [243.978, 17.414, -178.100, -115.487, 3.541, \\ 64.498, -91.178, -161.965, 115.029, -45.694, \\ 17.215, -2.594, -100.079, 87.252, 146.171]. \end{aligned}$$

#### 位置 26

$$w_{26} = [-0.2867, -0.0525, -0.8452, -0.4481]$$

$$c_{26} = [+,-,+,+,-,+,+,-,+,+,-,+,+,-,+]$$

$$\begin{aligned} z_{26} \approx [8.190, -76.789, 70.660, 83.474, -224.310, \\ 118.463, -4.068, 212.320, -198.156, 58.092, \\ -221.409, 162.834, -100.011, 65.344, 45.366]. \end{aligned}$$

这些码不是人工设计的 Hadamard 码，而是由三个位置的真实响应自动产生的。

## 4 发送端如何把 bit 编成 vector hue

### 4.1 从 bit 到符号组合

每个发送块包含 8 路符号：前三路是真实用户，后五路是虚拟流。记为

$$\mathbf{b} = (b_1, b_2, \dots, b_8), \quad b_j \in \{-1, +1\}.$$

第  $t$  个 Probe 时隙的符号组合为

$$\mathbf{s}[t] = (b_1 c_1[t], b_2 c_2[t], \dots, b_8 c_8[t]).$$

其中虚拟流复用真实位置的码，代码中由 `full_codes_for_virtual` 完成。

### 4.2 什么是 vector hue

普通 hue mapping 是

$$M : \{-1, +1\}^k \rightarrow q,$$

即一个符号组合对应一个 hue。当前实验使用 vector hue mapping:

$$M : \{-1, +1\}^8 \rightarrow (q_1, q_2, q_3, q_4),$$

四个分量分别送给四个 chip。也就是说，在同一个时隙内，chip1 可以使用 hue 100，chip2 可以使用 hue 215，chip3 可以使用 hue 75，chip4 可以使用 hue 295。

### 4.3 Hue mapping 如何计算

对某个符号组合  $\mathbf{a} = (a_1, \dots, a_8)$ ，代码先看真实合法位置的前三个符号。对每个 chip  $d$  和候选 Probe  $q$ ，计算得分

$$\text{score}_d(q) = \sum_{i=1}^3 a_i R_{\ell_i}[q, d] w_{\ell_i}[d].$$

得分越高，说明这个 chip 选择该 hue 后越能推动合法位置朝目标符号方向变化。每个 chip 取高分候选，再组合成四维 vector hue。代码位置为 `_build_vector_candidate_dict`。

### 4.4 真实计算例子：符号 $[-1, -1, -1, -1, -1, -1, -1, -1]$

候选 267 对该符号选出的 vector hue 为

$$M([-1, -1, -1, -1, -1, -1, -1, -1]) = (100, 215, 75, 295).$$

它不是手工指定的，而是按上面的得分公式计算得到。四个 chip 的真实得分如下。

| chip | 选择 hue | 位置 2 贡献  | 位置 5 贡献  | 位置 26 贡献 / 合计      |
|------|--------|----------|----------|--------------------|
| 1    | 100    | -7.3962  | 106.0635 | 46.0768 / 144.7442 |
| 2    | 215    | 88.2653  | -3.8111  | 9.6599 / 94.1141   |
| 3    | 75     | 113.7853 | 0.3549   | 73.1553 / 187.2955 |
| 4    | 295    | 52.1155  | 34.6001  | -14.8979 / 71.8178 |

四个 chip 合计得分约为

$$144.7442 + 94.1141 + 187.2955 + 71.8178 = 497.9716.$$

这说明该 vector hue 对目标符号方向总体是有利的。

## 5 完整发送和合法解码算例

### 5.1 一个真实发送块

从代码生成的第一个 block 为

$$\mathbf{b} = (-1, -1, -1, -1, -1, -1, -1, -1),$$

对应二进制为

$$[0, 0, 0, 0, 0, 0, 0, 0].$$

前三路真实用户的目标 bit 都是 0，也就是  $(-1, -1, -1)$ 。

前五个时隙的符号组合和发送 vector hue 如下。

| 时隙 | 符号组合 $\mathbf{s}[t]$               | 发送 vector hue          |
|----|------------------------------------|------------------------|
| 1  | $[-1, -1, -1, -1, -1, -1, -1, -1]$ | $[100, 215, 75, 295]$  |
| 2  | $[1, -1, 1, 1, -1, 1, 1, -1]$      | $[225, 25, 250, 250]$  |
| 3  | $[-1, 1, -1, -1, 1, -1, -1, 1]$    | $[355, 325, 315, 340]$ |
| 4  | $[1, 1, -1, 1, 1, -1, 1, 1]$       | $[25, 25, 315, 340]$   |
| 5  | $[-1, -1, 1, -1, -1, 1, -1, -1]$   | $[325, 215, 115, 225]$ |

注意：所有合法设备和非法设备接收到的是同一串公共 vector hue 序列。差别只在于每个位置的实测响应矩阵不同，所以同一发送序列在不同位置会变成不同观测。

### 5.2 合法设备的本地解码公式

位置  $p$  收到一个 block 后，先根据自己的真实响应矩阵查表，得到本地观测

$$Y_{p,\text{obs}} \in \mathbb{R}^{15 \times 4}.$$

在线解码时并不知道发送端真实 bit，因此不能减去已知趋势，只做块内中心化：

$$\tilde{Y}_p = Y_{p,\text{obs}} - \bar{Y}_{p,\text{obs}}.$$

然后投影：

$$u_p = \tilde{Y}_p w_p.$$

最后与本地地址码相关：

$$\gamma_p = c_p^\top u_p.$$

判决规则为

$$\hat{b}_p = \begin{cases} +1, & \gamma_p > 0, \\ -1, & \gamma_p \leq 0. \end{cases}$$

### 5.3 真实合法解码结果

对上面的 block，三个合法位置的真实计算结果如下。

| 合法位置 | 目标符号 | $u_p$ 前五项                                          | $\gamma_p$ | 判决    |
|------|------|----------------------------------------------------|------------|-------|
| 2    | -1   | $[-251.460, 149.749, 39.678, 145.117, -232.389]$   | -1771.5279 | -1 正确 |
| 5    | -1   | $[-102.894, -165.288, 164.176, 188.337, -173.011]$ | -2050.0303 | -1 正确 |
| 26   | -1   | $[-53.007, 275.969, -188.897, -149.776, 134.064]$  | -2310.4931 | -1 正确 |

因为三个  $\gamma_p$  都小于 0，三个合法设备都判为 -1，与发送目标完全一致。

## 6 非法设备为什么解码失败

### 6.1 非法设备也可以尝试同样算法

非法位置 25 也有自己的响应矩阵  $Y_{25}$ 、读出方向  $w_{25}$  和地址码  $c_{25}$ 。它可以把同一串 vector hue 当作自己的观测来解码：

$$\gamma_{25} = c_{25}^\top (Y_{25, \text{obs}} - \bar{Y}_{25, \text{obs}}) w_{25}.$$

但问题是：发送端的 mapping 是为了合法位置 (2, 5, 26) 的指纹结构搜索出来的，不是为位置 25 搜索出来的。因此位置 25 的输出不会稳定等于任意一路合法 bit。

### 6.2 单个 block 不足以说明安全，BER 才说明问题

在第一个 block 中，非法位置 25 的计算为

$$u_{25} \text{ 前五项} \approx [58.240, 44.477, -1.939, 0.233, -60.098],$$

$$\gamma_{25} \approx -460.6195,$$

所以它也判为 0。这个 block 正好和合法 bit 一样，不能说明泄露，也不能说明安全。安全性要看 1000 个随机 block 中，它和每一路真实 bit 的误码率。

对候选 267，在 1000 个真实随机 block 上，非法位置 25 的结果为：

| 非法路由                | raw BER | corrected BER |
|---------------------|---------|---------------|
| 25 $\rightarrow$ 2  | 0.424   | 0.424         |
| 25 $\rightarrow$ 5  | 0.449   | 0.449         |
| 25 $\rightarrow$ 26 | 0.447   | 0.447         |

这些值都接近 0.5，表示非法位置 25 对三路真实信息基本接近随机猜测。

## 7 Global Guarded 搜索流程

### 7.1 第一阶段：baseline 候选

`run_global_guarded_experiment` 先随机抽样合法位置组合。对每个组合，生成多个 Probe 子集。每个 Probe 子集加载合法位置模型，再生成多个 hue mapping 候选。每个候选都必须满足：

$$\text{BER}_{\text{authorized}} = 0.$$

也就是合法端在 1000 个 block 上不能出错。只有合法端完全正确的候选才进入安全评价。

### 7.2 第二阶段：找 anchor 和 weak routes

baseline 候选中，代码选出最好的 anchor：

$$\text{anchor} = \arg \max(\min \text{sBER}_{\text{route}}, \text{avg sBER}_{\text{route}}).$$

然后找出该 anchor 最薄弱的若干非法路由，称为 weak routes。对本例，baseline 阶段记录为：

| 字段                        | 真实值                                          |
|---------------------------|----------------------------------------------|
| 合法位置                      | (2, 5, 26)                                   |
| baseline anchor candidate | 34                                           |
| anchor 最小安全 BER           | 0.26500000                                   |
| anchor 最坏路由               | 20 → 5                                       |
| weak routes               | 20→5, 24→5, 9→5, 22→5, 22→2, 28→2, 27→2, 7→5 |

这些 weak routes 告诉程序：当前最危险的是哪些非法位置试图恢复哪些合法信息。

### 7.3 第三阶段：targeted repair

代码随后进行 `anchor_worst_targeted` 搜索。它继续生成更多 Probe 子集和 mapping 候选，重点寻找能改善 weak routes 的候选。候选 267 就来自该阶段，来源字段为 `anchor_worst_targeted`，Probe 子集编号为 27。

### 7.4 第四阶段：选择 20 个候选

`select_global_guarded_candidates` 不只选全局最强候选，而是组合四类候选：

- (1) anchor 本身，保证不低于一个已知强基准；
- (2) 全局表现最好的候选；
- (3) 对 weak routes 平均改善最大的候选；
- (4) 对单条 weak route 有救援能力的候选；
- (5) worst route 分布不同的候选，用来增加多样性。

这样做的原因是：单个候选可能某些路由很强、某些路由很弱；多个候选轮换使用后，非法设备很难长期稳定利用同一个弱点。

### 7.5 第五阶段：线性规划求 usage ratio

假设最终选出  $n = 20$  个候选，每个候选在每条非法路由上有一个 raw BER。把这些值组成矩阵

$$A \in \mathbb{R}^{R \times 20}.$$

代码用线性规划寻找使用比例

$$\mathbf{r} = (r_1, \dots, r_{20}), \quad r_j \geq 0, \quad \sum_j r_j = 1,$$

使混合后的最坏安全 BER 尽量大。混合 raw BER 为

$$\mathbf{e} = A\mathbf{r}.$$

安全 BER 使用 corrected BER:

$$\text{sBER} = \min(\text{BER}, 1 - \text{BER}).$$

这是因为如果非法端 BER 等于 1，它只要取反就能完全恢复，所以 BER=1 和 BER=0 一样危险。

## 8 本例最终结果

本例来自 `vector_hue/k3/global_guarded/results_summary.csv` 第一个样本，真实结果如下。

| 字段                        | 真实值                                  |
|---------------------------|--------------------------------------|
| real k                    | 3                                    |
| effective k               | 8                                    |
| virtual stream count      | 5                                    |
| 合法位置组合                    | (2, 5, 26)                           |
| 选中候选数                     | 20                                   |
| 共同非法路由数                   | 48                                   |
| 合法端最大 BER                 | 0.00000000                           |
| 三个合法位置 BER                | [0.00000000, 0.00000000, 0.00000000] |
| 混合后最小安全 BER               | 0.43411580                           |
| 混合后平均安全 BER               | 0.46757386                           |
| 最终最坏路由                    | 25 $\rightarrow$ 26                  |
| 优化器                       | linprog                              |
| anchor candidate          | 267                                  |
| anchor source             | anchor_worst_targeted                |
| anchor 最小安全 BER           | 0.32300000                           |
| security gain over anchor | 0.11111580                           |



最终 schedule 长度为 1000, 20 个候选的使用次数为:

[240, 156, 0, 0, 274, 0, 0, 35, 84, 0, 0, 0, 0, 73, 0, 13, 0, 125, 0, 0].

这表示实际发送时不是永远使用一个 mapping, 而是在 1000 个 block 中按上述次数轮换候选。这可以把最坏非法路由从 anchor 单独使用时的 0.323 提高到全局混合后的 0.43411580, 更接近理想随机猜测的 0.5。

## 9 完整流程总结

- (1) 读取所有位置的真实 CSV 响应数据。
- (2) 随机抽样合法位置组合, 例如 (2, 5, 26)。
- (3) 为该组合生成多个 15-Probe 子集。
- (4) 对每个合法位置、每个 Probe 子集做线性去趋势和 SVD, 得到  $w_p, z_p, c_p$ 。
- (5) 生成真实 bit 和虚拟 bit, 形成 8 维符号组合。
- (6) 根据残差、读出方向和目标符号, 为每个符号组合搜索四通道 vector hue mapping。
- (7) 用真实响应矩阵模拟合法端接收; 合法端必须 1000 个 block 全部正确, BER 为 0。
- (8) 用所有非法位置的响应矩阵模拟攻击; 计算每个非法位置到每路合法 bit 的 raw BER 和 corrected BER。
- (9) 从 baseline 候选中找 anchor 和 weak routes。
- (10) 对 weak routes 做 targeted repair, 继续搜索能修复弱路由的候选。
- (11) 从所有合法候选中选出 20 个互补候选。
- (12) 用线性规划求 usage ratio, 使混合后的最坏非法安全 BER 最大。
- (13) 输出 results\_summary.csv、results\_selected.csv 和 weak\_routes.csv。

## 10 结论

本实验已经从 PDF 1 的两位置最小闭环扩展为更强的安全验证流程。真实数据样本 (2, 5, 26) 表明:合法端三路 BER 均为 0;非法端最坏路由经过全局候选混合后 corrected BER 达到 0.43411580, 已经接近随机猜测。关键点在于, 地址码来自位置实测响应, vector hue mapping 由残差和 SVD 指纹计算得到, global guarded 机制再针对最危险非法路由做补强。